

高级语言程序设计大作业实验报告

Emoji_Dungeon

高级语言程序设计大作业实验报告项目名称：Emoji Dungeon
—— 基于 Qt 6.10 + C++17 的 Emoji 风格俯视角射击生存游戏
完成日期：2026 年 5 月 13 日

项目	内容
姓名	王雯霏
学号	2513883
日期	2026 年 5 月 13 日

目录

- 一、作业题目
- 二、开发软件
- 三、课题要求
- 四、主要流程
 - 4.1 整体流程
 - 4.2 算法或公式
 - 4.3 开发过程中遇到的难点及解决方法
- 五、测试情况
- 六、收获
- 七、实际进度安排
- 八、版本迭代历史
- 九、AI 工具使用披露
 - 9.1 使用工具总览
 - 9.2 按版本划分的 AI 使用明细

- 9.3 各 AI 工具的分工说明
- 9.4 文档撰写
- 9.5 人工主导部分声明

一、作业题目

Emoji Dungeon —— 基于 Qt 6.10 + C++17 的 Emoji 风格俯视角射击生存游戏。

本项目从零构建一款完整可运行、架构清晰、具备可扩展性的俯视角射击生存游戏。游戏采用 Rogue-lite 玩法范式，玩家选择职业后在 10 波敌人浪潮中生存，通过击败敌人获取经验、升级选择特性与武器强化，抵御不断增长的动态难度压力。

项目地址：<https://github.com/WwPei/my-nankai26C-project.git>

B 站视频地址：【【南开大学 26C++】？！emoji 的奇妙冒险！？】
https://www.bilibili.com/video/BV1gz5Q6BEZz/?share_source=copy_web&vd_source=3279b9403824cf662172bc5ee88c0e9c

二、开发软件

项目	工具/技术
开发环境	Qt Creator 16.0 + Visual Studio Code
GUI 框架	Qt 6.10.2 (Widgets + Graphics View)
编译器	MinGW 11.0 (64-bit)
构建系统	CMake 3.29+
C++ 标准	C++17
多媒体后端	FFmpeg（Qt Multimedia 模块）
版本管理	Git + GitHub
美术资源	Microsoft Fluent Emoji（MIT License）
AI 辅助工具	豆包、DeepSeek-V4、ChatGPT 5.4（通过

	Trac AI IDE 接入，详见第九节）
--	-----------------------

三、课题要求

3.1 问题描述

随着 Rogue-lite 类游戏（如《吸血鬼幸存者》《Hades》）在独立游戏市场的持续走红，“短局高密度决策”的游戏范式受到广泛关注。这类游戏通常需要玩家在有限时间内，通过职业选择、升级决策和操作技巧来抵御不断增长的外界压力。

作为 C++ 课程大作业，本项目目标为：

从零构建一款完整可运行、架构清晰、具备可扩展性的俯视角射击生存游戏，完整实践面向对象程序设计、多态、工厂模式、MV 分离架构等核心 C++ 知识点。

3.2 核心问题

设计并实现一个具备以下特征的交互式游戏系统：

- 1. **实时战斗系统**：支持玩家移动、射击、冲刺，与多类型敌人实时交互
- 2. **动态难度体系**：随波次递增，敌人血量、伤害按指数增长（ $HP \times 1.05^{wave}$ ， $DMG \times 1.03^{wave}$ ）
- 3. **数据驱动的游戏配置**：所有职业、武器、敌人、特性的数值通过静态配置表管理，分离逻辑与数据
- 4. **完整的升级决策链路**：击败敌人获取经验 → 升级，随机获得 5 种特性/武器强化或特殊子弹选择
- 5. **多 AI 行为体系**：6 种差异化 AI 行为模式（追逐、射击游击、蓄力冲锋、自爆、Boss 多阶段等）
- 6. **工程化架构**：MV 分离、工厂模式、信号槽解耦、CMake 多目标构建

3.3 功能需求总览

功能模块	具体内容
职业系统	战士（120 HP / 235 速度）、游侠（95 HP / 270 速度）、施法者（85 HP / 220 速度），各有专属武器

敌人生成	10 种敌人（含 4 Boss），分属怪异、亡灵、外星 3 个系列，波次递进解锁
6 种 AI	Chase / KeepDistance / Charge / ShootAndMove / SuicideBomb / Boss
武器系统	豆豆发射器（平行双发）、散射喷发器（扇形三发）、弧光法杖（单发电弧）+ 5 种武器强化
升级系统	16 级经验阈值表 + 10 种特性 + 5 种武器强化 + 12 种特殊子弹，每级随机 5 选 1
特殊子弹	12 种（飞刀/太阳法球/月亮法球/哈基米追踪猫/雷霆之矛/回力镖/血之箭/眩晕弹/随机礼物/彗星/击退弹/火箭）
冲刺系统	空格键触发，3 秒冷却，UI 环形冷却指示器
音效与 BGM	6 首 BGM 按波次与 Boss 切换，2 种音效（普通攻击命中/击杀）
动态面部表情	玩家角色随血量变化切换 6 种表情（微笑/嘘声/大哭/光环/大笑/生气）
UI/HUD	悬浮半透明面板（HP/经验/冲刺 CD/特性/武器）、Boss 固定血条、升级弹窗动画

四、主要流程

4.1 整体流程

4.1.1 设计思路

项目从一开始就确立了“先骨架、后血肉、再打磨”的渐进式开发思路：

阶段 0（架构先行）：

- 在写任何游戏逻辑之前，先建立 8 个纯虚抽象接口和 GameFactory 工厂
- 确定 QGraphicsView + QGraphicsScene 渲染方案和 QStackedWidget 页面管理方案
- 确立“页面只负责导航，游戏对象只负责状态变化通知”的信号槽约定
- 所有接口方法标 `[[nodiscard]]`，编译期约束返回值不可忽略

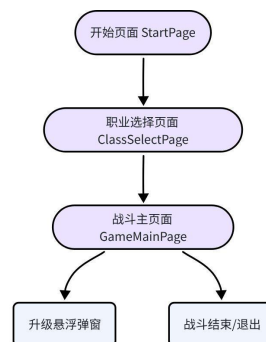
阶段 1-2（功能实现）：

- 从接口出发，逐步实现各 Basic* 类的具体逻辑
- 建立 16ms 主循环驱动全游戏帧更新
- 按照“数据驱动”原则，所有游戏数值从 GameConfig 配置表读取，不在代码中硬编码

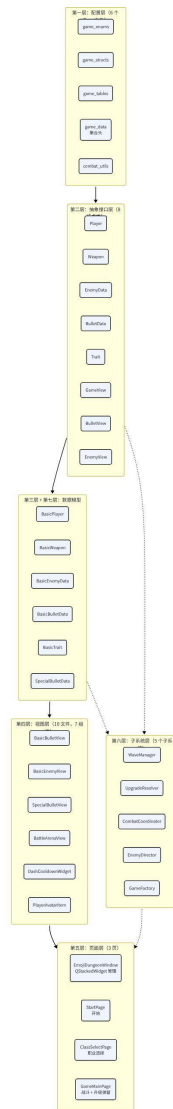
阶段 3-4（打磨优化）：

- UI 层面从功能优先转向体验优先（悬浮 HUD、动画弹窗、动态表情）
- 数值层面降低经验门槛、提高怪物密度，让游戏节奏更快更爽
- 视听层面接入完整 BGM 和音效系统

4.1.2 游戏主流程



4.1.3 7 层总体架构

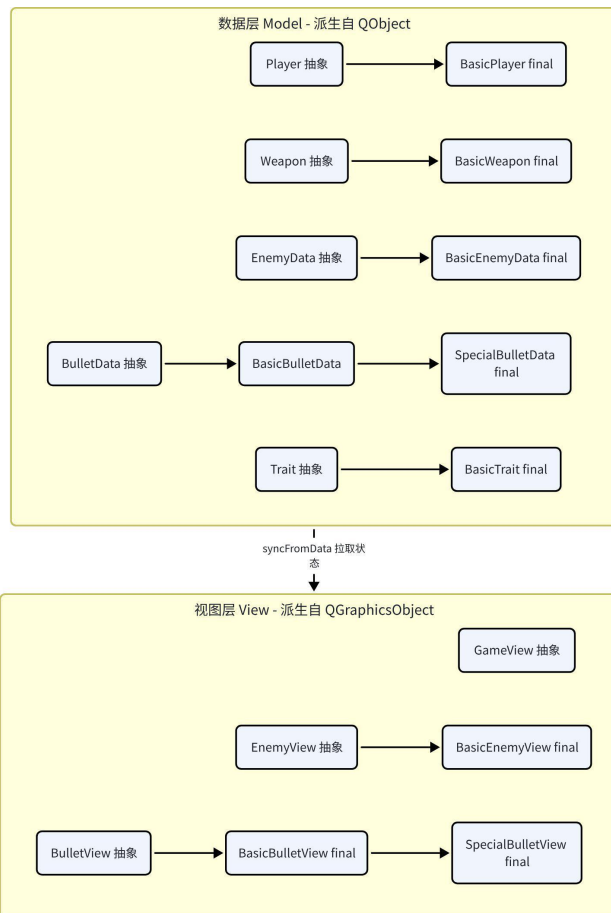


层级依赖规则（严格遵守）：

- 类型一被所有其他类型依赖，不依赖任何类型
- 类型二的实现由类型三完成（final 封闭继承）
- 类型四的模型绑定通过类型二的接口指针完成（依赖倒置原则）
- 类型五通过类型六的子系统操作类型四的 View 对象，通过类型二接口操作类型三的 Model 对象
- 类型七是类型三的子类型（SpecialBulletData 继承 BasicBulletData）

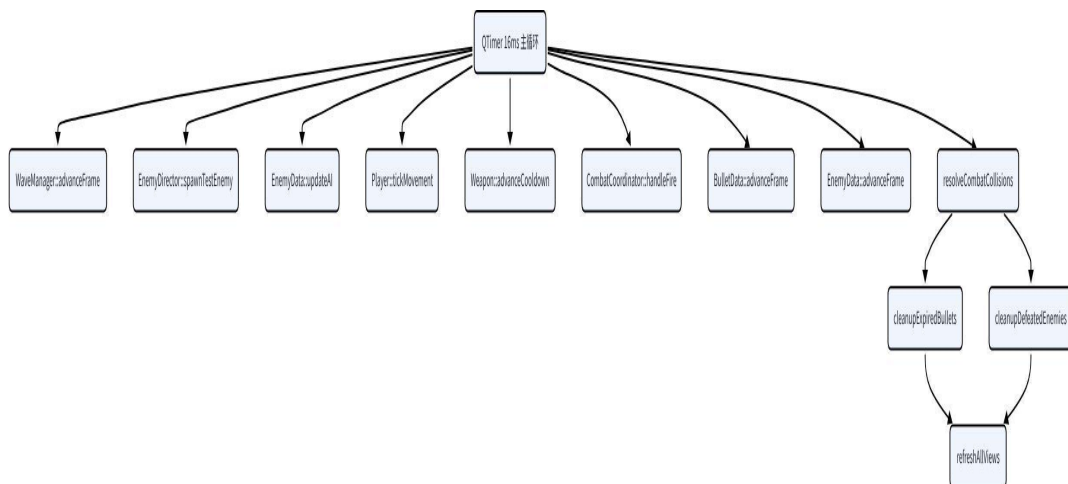
4.1.4 MV 分离设计

项目最核心的架构原则是 **Model-View 分离**：



View 层通过 `syncFromData()` 纯虚方法每帧从 Model 拉取最新状态，不持有 Model 的数据拷贝，仅持有 `QPointer<ModelInterface>` 安全引用。

4.1.5 数据流架构



4.1.6 文件组织

目录	文件数	职责
src/core/	6	枚举定义、配置结构体、静态数据表、工具函数
src/models/interfaces/	6	数据模型纯虚抽象基类
src/models/basic/	10	Basic* 具体实现类（全部 final）
src/views/interfaces/	2	视图抽象基类
src/views/	10	视图实现 + BattleArenaView + DashCooldownWidget + PlayerAvatarItem
src/pages/	6	页面层： EmojiDungeonWindow / ClassSelectPage / GameMainPage
src/systems/	10	GameFactory / WaveManager / UpgradeResolver / CombatCoordinator / EnemyDirector
src/bullets/	5	SpecialBulletData + SpecialBulletView + 12 种模板配置
resources/	82	73 张 PNG + 8 首 MP3 + 1 QRC

4.2 算法或公式

4.2.1 敌人 AI 行为算法

6 种 AI 行为通过 BasicEnemyData 中的 switch-case 统一处理，核心是以“与玩家距离”为

决策依据的有限状态机。

Chase（追逐）—— SkeletonNew 使用：

Plain Text

每帧: `setTargetPosition(playerPosition)` → 直线移动

KeepDistance（保持距离）—— 预留，未分配：

Plain Text

if (与玩家距离 < `keepDistanceMin`):

 朝远离玩家方向移动

else:

 向玩家靠近到 `keepDistanceMin` 距离

Charge（蓄力冲锋）—— Ogre、XenoBeast 使用，三段流程：

Plain Text

Phase 1 — 追逐: 向玩家移动

Phase 2 — 蓄力: CD 到达后原地不动 `chargeUpMs`（Ogre: 800ms / XenoBeast: 500ms）

Phase 3 — 冲锋: 以 `velocity × chargeSpeedMult` 冲向玩家 `chargeDurationMs`

Phase 4 — 冷却: 进入 `chargeCooldownMs` 冷却（Ogre: 3s / XenoBeast: 2.5s）

ShootAndMove（射击游击）—— Jester、Ghost、Robot 使用：

Plain Text

if (与玩家距离 $\geq$ `keepDistanceMin × 85%`):

 向玩家移动

elif (距离 < `keepDistanceMin × 85%`):

 朝远离玩家方向移动

if (`shootCooldown` $\leq$ 0):

 向玩家方向发射 `shootBulletCount` 发子弹，重置 CD

SuicideBomb（自爆）—— 预留，未分配：

Plain Text

始终向玩家移动，距离 $\leq$ `suicideRadius` 时引爆造成范围伤害

Boss（多阶段首领）—— DemonLord、BoneLord、UFO、AlienPilot 使用：

Plain Text

Phase 1: 始终向玩家移动 + CD 到达时发射圆弧弹幕 (shootBulletCount 发子弹)

Phase 2 (HP < 50%): 移动速度 $\times 1.5$, 射击 CD 重置

spawnCooldown ≤ 0 时召唤当前系列的小兵

动态难度公式:

Plain Text

enemyHpActual = baseHp $\times 1.05^{\text{wave}}$

enemyDmgActual = baseDmg $\times 1.03^{\text{wave}}$

expMultiplier = 1.0 (wave 1-3)

= 1.1 (wave 4-7)

= 1.2 (wave 8-9)

= 1.5 (wave 10+)

4.2.2 武器射击算法

Plain Text

输入: weaponId, origin (发射位置), direction (朝向)

1. 查 WeaponConfig 获取 projectileCount 和 bulletStyle
2. 根据 bulletStyle 创建普通弹或特殊弹 (SpecialBulletData)
3. 并行弹道计算:
 - PeaShooter (豆豆发射器): 使用平行偏移 kParallelOffset = 10px
 - SpreadBlaster (散射喷发器): 使用旋转扩散 kSpreadStepDegrees = 12°
 - ArcWand (弧光法杖): 单发直线
4. extraProjectiles (武器强化): 在基础弹道两侧追加额外子弹
5. 注入武器强化参数: pierceCount, bulletSizeScale, damageMultiplier, comboMultiplier
6. 场景添加 BulletView $\rightarrow$ 推入 bullets 向量

4.2.3 碰撞检测算法

Plain Text

resolveCombatCollisions():

// 玩家子弹 vs 敌人

for each playerBullet:

collidingItems = bullet.view $\rightarrow$ collidingItems()

```
for each collided:
    if (item is EnemyView):
        if (trait == Frostbite): enemy → applySlow(40%, 2s)
        if (trait == CriticalStrike && random() < 15%): damage *= 1.5
        enemy → receiveDamage(damage)
        if (pierceCount > 0): pierceCount-- (continue)
        else: mark bullet expired (break)

// 敌人子弹 vs 玩家
for each enemyBullet:
    if collide with playerMarker:
        player → receiveDamage(damage)

// 近战接触伤害
maxContactDamage = max(enemy.contactDamage)
if (maxContactDamage > 0 && playerDamageCooldown ≤ 0):
    player → receiveDamage(maxContactDamage)
    playerDamageCooldown = 600ms
```

4.2.4 经验与等级系统算法

Plain Text

经验阈值公式: $threshold[n] = \sum(10 \times i^2), i = 1..n-1$

第 n 级升级需要: $10 \times (n-1)^2$ 经验值

升级判定: levelForExperience(exp) → 二分查找阈值表

经验获得: $enemyBaseExp \times waveExpMultiplier \times playerExpMultiplier$

升级触发: 从 kAllUpgradeOptions 随机抽取 5 个（已拥有则自动升级）

特性叠加: QuickHands Lv2 → $damage \times 1.15^2, speed \times 1.05^2$ （乘法叠加）

4.2.5 核心技术选型

技术决策	选择方案	理由
渲染引擎	QGraphicsView + QGraphicsScene	天然适配大量动态对象、碰撞检测、多图层管理
页面管理	QStackedWidget（3 页）	页面职责清晰，切换简洁

升级 UI	悬浮弹窗 + QVariantAnimation	重叠于战斗画面，避免整 页切换撕裂感
对象创建	GameFactory 工厂模式	统一入口，基于配置 ID 实 例化
碰撞检测	collidingItems() + HP 比较 双重校验	确保碰撞可靠性
主循环	QTimer(16ms) ≈ 60FPS	固定帧率驱动，保证游戏 节奏统一
音频方案	QMediaPlayer + QAudioOutput	兼容 FFmpeg MP3 解码
配置管理	编译期静态表 (inline const)	零运行时开销，所有数据 编译期确定

4.3 开发过程中遇到的难点及解决方法

难点一：QSoundEffect MP3 解码不兼容

问题：使用 QSoundEffect 播放 .mp3 命中音效时，控制台持续报 Error decoding source，音效无法播放。

根因：Qt 6.10 的 FFmpeg 多媒体后端与 QSoundEffect 的 MP3 解码管道不兼容，QSoundEffect 内部使用独立解码路径，仅稳定支持 .wav 格式。

解决方法：彻底移除 QSoundEffect，统一改用 QMediaPlayer + QAudioOutput。两者使用同一 FFmpeg 后端，MP3 兼容稳定。同时简化了音效管理（无需分别维护两套音频对象）。

```
C++
// 修改前（不兼容）
QSoundEffect *hitSound = new QSoundEffect(this);
hitSound->setSource(QUrl("qrc:/musics/hit.mp3")); // Error decoding source

// 修改后（兼容）
QMediaPlayer *hitPlayer = new QMediaPlayer(this);
QAudioOutput *audioOutput = new QAudioOutput(this);
```

```
hitPlayer->setAudioOutput(audioOutput);  
hitPlayer->setSource(QUrl("qrc:/musics/hit.mp3"));
```

难点二：升级弹窗渲染崩溃

问题：战斗页面切换到升级时程序崩溃，堆栈指向 `QGraphicsOpacityEffect::draw()`。

根因：`QGraphicsOpacityEffect` 与 `QGraphicsView` 的 GPU 渲染管道冲突。`QGraphicsView` 内部使用独立的绘制路径，`QGraphicsOpacityEffect` 尝试在 `QWidget` 层级重定向绘制目标时与 `QGraphicsView` 的 viewport 绘制发生冲突。

解决方法：用 `QVariantAnimation` + `QSS background: rgba(r, g, b, alpha)` 动态改变 `alpha` 通道值实现淡入淡出效果。动画从 `rgba(10,12,18,0)` 过渡到 `rgba(10,12,18,0.88)`（淡入 350ms OutCubic），反过来淡出 280ms InCubic。

```
C++  
// 废弃方案（会崩溃）  
QGraphicsOpacityEffect *effect = new QGraphicsOpacityEffect(this);  
upgradeOverlay->setGraphicsEffect(effect);  
  
// 最终方案（稳定）  
QVariantAnimation *fadeIn = new QVariantAnimation(this);  
fadeIn->setDuration(350);  
fadeIn->setEasingCurve(QEasingCurve::OutCubic);  
fadeIn->setStartValue(0);  
fadeIn->setEndValue(0.88);  
connect(fadeIn, &QVariantAnimation::valueChanged, this, [this](const QVariant &v) {  
    int alpha = qRound(v.toDouble() * 255);  
    upgradeOverlay->setStyleSheet(  
        QString("background: rgba(10,12,18,%1);").arg(alpha));  
});
```

难点三：升级弹窗事件捕获冲突

问题：使用 `qApp->installEventFilter(this)` 全局捕获点击事件实现“点击弹窗外部关闭”时，升级弹窗内部按钮点击无响应。

根因：全局事件过滤器优先级高于子控件事件分发，会将所有鼠标事件在分发到子控件之前拦截。

解决方法：删除 `qApp` 事件过滤器，改为在每个升级卡片上 `installEventFilter`，通过 `property("cardIndex")` 独立捕获点击。弹窗外部点击通过遮罩层的 `mousePressEvent` 处理。

难点四：角色/子弹拖尾残影

问题：游戏运行中角色移动和子弹飞行出现拖尾残影，画面脏区域未正确重绘。

根因：`QGraphicsView` 默认使用 `MinimalViewportUpdate` 模式，只重绘被标记为脏的矩形区域。在高密度对象（22 个敌人 + 大量子弹同时存在）场景下，增量脏区域标记不够精确，导致上一帧的部分像素未被覆盖。

解决方法：设置 `setViewportUpdateMode(QGraphicsView::FullViewportUpdate)`，改为每帧全视口重绘。虽然理论上性能开销更大，但实测在 60FPS 下帧率稳定，且彻底解决了残影问题。

C++

```
// battle_arena_view.cpp 构造函数中
setViewportUpdateMode(QGraphicsView::FullViewportUpdate); // 单行修复
```

难点五：WASD 在升级界面仍然生效

问题：升级弹窗显示时，玩家 WASD 移动仍然响应，导致角色在弹窗背后移动。

根因：旧流程先 `setBattleActive(false)` 停止战斗循环，但未先设置 `BattleFlowState::Upgrade`。键盘事件处理函数中判断 `battleState != BattleFlowState::Active` 时才拦截，由于状态未更新，WASD 继续生效。

解决方法：改为先调用 `setBattleState(BattleFlowState::Upgrade)` 再 `setBattleActive(false)`，确保键盘事件按 `battleState` 正确拦截。

C++

```
// 修改前
setBattleActive(false); // 停止战斗循环
// battleState 仍为 Active → WASD 继续响应

// 修改后
setBattleState(BattleFlowState::Upgrade); // 先设置状态
setBattleActive(false); // 再停止循环
```

难点六：CMake + windres 路径空格编译失败

问题：项目路径 F:\QT projects\emoji_dungeon_v3 含空格，CMake 将 C++ include 目录路径传递给 windres.exe 编译 .rc 资源文件时，空格被误解析为参数分隔符。

根因：CMake 内部将 target_include_directories 的所有路径作为 -I 参数传递给 windres，但 windres 使用空格作为参数分隔，无法正确处理含空格的路径。

解决方法：手动先用 windres 编译 .rc → .o，再通过 CMake 的 target_link_libraries 直接链接预编译对象文件，绕过 CMake 的 include 路径传递。

```
CMake
# CMakeLists.txt 中的绕行方案
add_custom_command(
    OUTPUT app_icon_res.o
    COMMAND windres -i app_icon.rc -o app_icon_res.o
    DEPENDS app_icon.rc
)
target_link_libraries(emoji_dungeon PRIVATE
"${CMAKE_CURRENT_SOURCE_DIR}/app_icon_res.o")
```

难点七：Boss 血条每帧抖动

问题：Boss 血条跟随 Boss 场景坐标移动时出现抖动。

根因：每帧追踪 Boss 的 scenePos() 并映射到 viewport 坐标，在高帧率下场景坐标更新和 UI 重绘之间存在一帧延迟，导致血条位置在相邻帧之间微小跳动。

解决方法：Boss 血条改为固定在波次进度条正下方（positionBossBarFixed((w - 300) / 2, kHudMargin + 28)），不再追踪 Boss 场景坐标。Boss 血条始终在屏幕顶部居中，视觉效果更稳定。

难点八：背景图片不显示

问题：BattleArenaView 上设置的背景图片无法显示。

根因：背景图片在 paintEvent 中绘制，但 QGraphicsView 的子控件（viewport）覆盖了 paintEvent 的绘制结果。

解决方法：将背景图片绘制移至 BattleArenaView::drawBackground()，由 QGraphicsView 的 background 机制处理，确保背景在所有图形项之后、viewport 之前渲染。

难点九：机器人敌人不射击

问题：Robot（机器人）敌人不会发射子弹。

根因：shootTriggerDistance 字段被误设为 true（布尔值隐式转换为 1.0），与期望的射击触发距离语义不符，导致射击判定逻辑错误。

解决方法：将 shootTriggerDistance 改为 0.0F，使机器人在任意距离都能触发射击。

五、测试情况

5.1 测试策略

由于本项目是图形化游戏应用，传统单元测试框架（如 Qt Test）难以覆盖实时渲染、物理碰撞和用户交互等核心场景。项目采用以下测试策略：

测试类型	方法	覆盖内容
编译期验证	[[nodiscard]] + enum class + final + inline const	类型安全、接口契约、继承链封闭
运行时断言	Qt 运行时警告 + 控制台日志	空指针检测、资源加载失败、信号槽连接异常
手动功能测试	逐项功能验证	职业选择、战斗循环、AI 行为、升级流程、音效 BGM
边界测试	极端数值测试	最大波次 (10)、最高等级 (16)、最大同屏敌人 (22)、满级全特性叠加
回归测试	每版本全流程跑通	V1→V4 每个版本交付前完整通关测试

5.2 关键功能测试用例

编号	测试项	输入/条件	预期结果	实际结果
----	-----	-------	------	------

1	三职业创建	依次选择战士/游侠/施法者	HP=120/95/85 ， 速度 =235/270/220	通过
2	WASD 移动	按下 W/A/S/D	角色按方向移动，不超出场景边界	通过
3	鼠标瞄准	移动鼠标	武器图标跟随鼠标方向旋转	通过
4	自动射击	武器 CD 归零	向鼠标方向发射子弹	通过
5	冲刺	按下空格	角色向前冲刺，CD 3 秒环形指示器正常	通过
6	敌人 Chase AI	SkeletonNew 生成	始终向玩家移动	通过
7	敌人 Charge AI	Ogre 生成	蓄力→冲锋→冷却三段流程	通过
8	敌人 ShootAndMove AI	Jester/Ghost/Robot 生成	边射击边保持距离	通过
9	Boss 多阶段	DemonLord HP<50%	速度×1.5，射击 CD 重置	通过
10	Boss 召唤小兵	BoneLord spawnCooldown≤0	召唤 SkeletonNew	通过
11	碰撞伤害	敌人碰撞玩家	玩家扣血，600ms 冷却防连续扣血	通过
12	升级触发	经验达到阈值	弹出升级弹	通过

			窗，显示 5 个选项	
13	升级弹窗动画	升级触发	350ms 淡入，选中后 280ms 淡出	通过
14	特性叠加	QuickHands Lv2	damage $\times$ 1.15 ² , speed $\times$ 1.05 ²	通过
15	冰冻减速	Frostbite 命中敌人	减速 40%，持续 2s	通过
16	弹射弹	BouncingBullet 命中	弹射到 180px 内最近敌人，保留 70% 伤害	通过
17	暴击	CriticalStrike 触发	15% 概率 1.5 倍伤害	通过
18	吸血	VampiricAura 击杀	回复 10% 最大生命	通过
19	哈基米追踪	Hacimi 发射	追踪最近敌人 → 挂身持续伤害 3s	通过
20	眩晕弹	StunBullet 命中	目标眩晕 1.5s (不动不攻击)	通过
21	击退弹	PushBullet 命中	击退 150px, 撞墙 +30% 伤害	通过
22	BGM 切换	波次推进	sery1→sery1_boss→sery2→sery2_boss→ser3→ser3_boss	通过

23	面部表情切换	HP 变化	100%→>50%→<50%→≤30% 四种表情 + 冲刺/被击中	通过
24	动态难度	波次递增	HP×1.05^wave, DMG×1.03^wave	通过
25	满级全特性	Lv16 + 全特性叠满	游戏正常运行, 无崩溃	通过

5.3 版本稳定性测试

版本	测试结果	已知问题
V1	基础战斗循环正常	单体架构, 无 AI 行为, 仅有 3 种敌人
V2	所有新功能正常	升级页面偶有卡顿 (V4 修复)
V3	工程化重组后功能不变	无退化问题
V4	全流程通关测试通过	无已知阻塞性问题

六、收获

6.1 面向对象设计与多态

通过本项目深刻理解了抽象基类和多态的实际应用价值。项目定义了 8 个纯虚抽象接口 (Player、Weapon、EnemyData、BulletData、Trait、GameView、BulletView、EnemyView), 所有游戏实体都通过基类指针进行操作, 这样在添加新敌人类型或新子弹类型时无需修改战斗协调器等核心逻辑。

```
C++
// 多态的核心价值: 操作接口, 不关心具体类型
void CombatCoordinator::handleWeaponFireRequested(const QPointF &origin, const
```

```

QPointF &direction) {
    // m_playerWeapon 是 Weapon* 抽象指针，实际可能是
    PeaShooter/SpreadBlaster/ArcWand
    BulletConfig config = m_factory->bulletConfigFor(m_playerWeapon->bulletStyle());
    // 统一通过接口操作，不关心具体武器类型
}

```

6.2 虚析构函数的重要性

项目中所有抽象基类都声明了虚析构函数。这确保了通过基类指针删除子类对象时，子类的析构函数能被正确调用，避免内存泄漏。

```

C++
// player.h
class Player : public QObject {
    Q_OBJECT
public:
    virtual ~Player() = default; // 虚析构函数，确保子类正确析构
    // ...
};

```

6.3 MV 分离架构的价值

Model（数据）和 View（渲染）的彻底分离是项目中最重要架构决策。最初 V1 版本将数据和渲染逻辑混在一起，导致代码难以维护。V2 重构后实现了完整的 MV 分离：

- **Model 层**只关心数据状态变化（位置、血量、CD），不涉及任何渲染代码
- **View 层**只关心如何绘制（贴图、血条、特效），通过 `syncFromData()` 从 Model 拉取状态
- **两者通过信号槽通信**，Model 发出状态变化信号，View 或其他系统响应

这种分离使得：

7. 可以独立更换渲染方案（从 QPainter 纯色绘制到 PNG 贴图渲染，Model 代码零修改）
8. 可以独立测试数据逻辑（不依赖图形环境）
9. 代码职责清晰，单个文件行数控制在合理范围

6.4 工厂模式解决对象创建复杂性

GameFactory 工厂类集中管理所有游戏对象的创建，通过配置 ID 统一实例化。这使得：

- 添加新敌人/武器/特性只需在配置表中增加条目，工厂自动支持
- 对象创建逻辑集中在一处，避免各处散布构造函数调用
- 工厂返回接口指针，调用方不依赖具体实现类

6.5 编译期类型安全

项目中充分利用了 C++17 的编译期特性来防止运行时错误：

- `[[nodiscard]]`：标记所有不应忽略返回值的函数，编译器在返回值被忽略时产生警告
- `enum class`：13 个强类型枚举，避免隐式类型转换和命名空间污染
- `final`：所有 `Basic*` 实现类标记为 `final`，防止意外的继承扩展
- `inline const`：所有配置表编译期确定，零运行时开销

6.6 增量式开发的价值

项目经历了 V1→V2→V3→V4 四个版本，每次迭代都建立在上一版本稳定的基础上：

- **V1** 验证了技术可行性（Qt Graphics View + 游戏主循环）
- **V2** 在稳定架构上大幅扩充内容（从 3 种敌人到 10 种，新增 12 种特殊子弹）
- **V3** 在不改变逻辑的前提下工程化重组（10 个子目录，CMake Strategy A）
- **V4** 在内容完整的基础上打磨体验（悬浮 HUD、弹窗动画、BGM 系统）

这种渐进式开发避免了“大爆炸”式集成的风险，每次交付都有一个可运行、可演示的版本。

6.7 Git 版本管理的体会

V1→V2→V3→V4 四个版本均有完整的 CHANGELOG 记录和架构文档。每次提交都有明确的版本标识，能够清晰追溯每个功能的引入时间。当 V4 出现 Bug 时，可以通过 Git 历史快速定位问题是在哪个版本引入的，极大提高了调试效率。

七、实际进度安排

7.1 阶段划分

阶段	时间	交付内容
阶段 0：骨架搭建	第 1 周	Widgets 框架、QStackedWidget 页面占位、8 个纯虚接口声明、GameFactory 工厂接口、game_data.h 配置结构声明、阶段 0 架构设计文档
阶段 1：核心模型	第 2 周	BasicPlayer / BasicWeapon / BasicEnemyData / BasicBulletData 实现、QTimer(16ms) 主循环、碰撞检测接入
阶段 2：玩法完整	第 3 周	波次系统 + 敌人上限控制、6 种 AI 行为状态机、伤害结算、升级/特性应用链路、动态难度公式、12 种特殊子弹、BGM/音效系统
阶段 3：视听与打磨	第 4 周	UI 悬浮 HUD 重制、动画升级弹窗、6 种动态面部表情、武器瞄准 PNG 指示器、Bug 修复与数值调优
最终交付	第 5 周	代码清理、配置 Release 构建、嵌入 .exe 图标、windeployqt 分发包、项目文档完善

7.2 里程碑节点

日期	版本	里程碑	关键产出
04-28	V1	单体原型完成	3 敌人/3 特性/基础战斗循环/1,183 行 game_main_page.

			cpp
05-05	—	MV 分离框架完成	8 个抽象接口 + 具体 Basic 类通过 GameFactory 创建
05-08	V2	功能大爆发	敌人 3→10 / 特性 3→10 / 武器强化 5 / 特殊子弹 12 / AI 行为 6 / 冲刺系统
05-09	V3	工程化重组	扁平 1 层 → 分层 10 子目录 / CMake Strategy A / 测试独立编译
05-11	V4	视听打磨交付	UI 全屏 HUD / 升级弹窗动画 / 6 BGM / 动态表情 / 数值优化 / Bug 修复
05-13	—	最终版本	代码清理 / Exe 构建与分发 / 项目报告

7.3 最终项目规模

类型	文件数	估计行数
配置层 (Core)	6	~600
抽象接口 (Interfaces)	8	~250
模型实现 (Models/Basic + SpecialBullet)	15	~900
视图层 (Views)	10	~700
页面层 (Pages)	6	~1,200

子系统（Systems）	10	~900
特殊子弹（Bullets）	5	~200
资源文件	1 QRC + 73 PNG + 8 MP3	—
CMake 配置	1	~120
总计	57 源文件 + 82 资源文件	~4,870

7.4 游戏内容规模总览

维度	数量	维度	数量
职业	3	AI 行为模式	6
武器	3	特殊子弹	12
敌人类型（含 Boss）	10	最高等级	16
Boss 波次	4 波	最大波次	10
特性	10	怪物系列	3
武器强化	5	最大同屏敌人	22
升级选项总数	27	初始敌人数量	6
BGM 曲目	6	精灵/图标素材	73

八、版本迭代历史

8.1 版本演进总览

版本	日期	头文件	源文件	代码行数	关键里程碑
----	----	-----	-----	------	-------

V1	04-28	15	7	~2,700	单体架构：单文件配置 + 单文件战斗逻辑，3 敌人/3 特性
V2	05-08	35	22	~4,500	MV 分离 + Plan B 三模块拆分，10 敌人/10 特性/12 特殊子弹/6 AI
V3	05-09	35	22	~4,500	工程化重组：1 层扁平 → 3 层 10 子目录，CMake Strategy A
V4	05-11	33	20	~4,700	UI 全屏重制 + 悬浮 HUD + 升级弹窗动画 + 数值优化 + 视听系统

8.2 V1 → V2：功能大爆发

V2 是项目中变化最大、内容增长最多的版本。

架构层面：

- `game_data.h` 单体配置（393 行）拆分为 `game\_enums.h`（106 行）、`game\_structs.h`（137 行）、`game\_tables.h`（568 行）
- `game_main_page.cpp` 中 ~1,183 行逻辑委托给三个独立子系统：UpgradeResolver（171 行）、CombatCoordinator（343 行）、EnemyDirector（331 行）
- `game_factory.cpp` 从 859 行精简至 205 行，具体实现拆分到 `basic_*.cpp`

内容层面：

维度	V1	V2	变化
敌人数量	3	10（含 4 Boss，3 怪物系列）	+233%
特性数量	3	10	+233%
AI 行为	1（Chase）	6	+500%
武器强化	0	5	全新系统

特殊子弹	0	12	全新系统
等级上限	10	15	+50%
QRC 资源	0	47 张 PNG	全新系统
冲刺系统	无	空格键 + 3s CD	全新系统
动态难度	无	HP $\times 1.05^{\text{wave}}$, DMG $\times 1.03^{\text{wave}}$	全新系统

8.3 V2 → V3：工程化重组

V3 是架构规范化版本，代码逻辑几乎不变，但文件组织方式发生了根本性变化：从 src/ 单层 57 文件平铺 → 10 个子目录分层管理。CMake 升级为 Strategy A，启用 AUTOMOC/AUTORCC，测试独立编译。

8.4 V3 → V4：视听打磨与稳定性

V4 聚焦完整可玩性和视听表现：UI 全屏重制（边距 0px + 悬浮 HUD）、升级从整页切换到悬浮弹窗（QVariantAnimation 动画）、经验阈值降低约 25%、最大同屏敌人 15→22、敌人生成间隔 2,200ms→850ms、6 首 BGM 按波次和 Boss 切换、6 种动态面部表情、修复 7 个关键 Bug。

九、AI 工具使用披露

根据课程要求，以下披露本项目中使用 AI 工具的情况：

9.1 使用工具总览

项目	内容
AI 大模型	豆包、DeepSeek-V4、ChatGPT 5.4
AI IDE 平台	Trae（接入 ChatGPT 5.4 / DeepSeek-V4 模型）
具体用途	代码生成、重构与优化、架构设计咨

	询、Bug 排查与修复、资源批量导入、文档撰写、构建部署
--	------------------------------

9.2 按版本划分的 AI 使用明细

V1 版本（手动为主，AI 咨询为辅）

维度	内容
手动完成	手动创建所有源文件，编写基础战斗循环、敌人逻辑和页面框架
AI 使用	对豆包和 DeepSeek 进行技术咨询，辅助理解 Qt 框架 API 用法和 C++ 语法细节
遗留问题	多种功能在单一文件中高度耦合，维护和更新极其困难； 存在重复删除空指针的恶性 Bug，导致游戏异常崩溃且难以排查修复

V2 版本（AI 主导重构，手动实现核心逻辑）

维度	内容
AI 完成（Trae + ChatGPT 5.4）	1. 项目结构重构：将 V1 单体架构拆分为分层结构，引入 工厂模式 、 MV 分离架构 、 多态机制 ，解决了代码耦合问题 2. Bug 修复：定位并修复了 V1 遗留的 重复删除空指针 导致的崩溃 Bug；修复了鼠标信号在某些场景下无法正确连接到信号槽的无效响应问题 3. 玩家主控角色功能：实现了玩家移动、射击、冲刺等核心功能 4. 怪物波次管理：实现了 WaveManager 波次调度系统
手动完成	1. 战斗页面：GameMainPage 战斗编排器核心逻辑 2. 升级页面：UpgradePage 升级界面与选

	项系统 3. 基础敌对单位： basic_enemy_data.h/.cpp 敌人数据模型 4. AI 行为逻辑：6 种 AI 行为（Chase / KeepDistance / Charge / ShootAndMove / SuicideBomb / Boss）的伪代码设计到 C++ 实现 5. 简单 UI 规划：初始版本的状态面板和页面布局
--	--

V3 版本（AI 辅助批量操作，手动细化游戏内容）

维度	内容
AI 完成（Trae + DeepSeek-V4）	1. 美术资源注册：搜集大量 Fluent Emoji PNG 资源后，使用 Trae 创建模板并进行重复编码（批量注册到 QRC 文件） 2. 特殊子弹增加：基于之前创建的基础子弹模板，批量生成了 12 种特殊子弹的数据和视图代码 3. 删除基础怪物：使用 Trae 在多个文件中精准定位并删除 V1 遗留的 Slime/Bat/Skeleton 旧代码（无需手动逐个文件查找） 4. 文件分类整合：对 src/ 目录下全部 57 个文件按职责分类到 core/models/interfaces/basic/views/pages/systems/bullets 子目录 5. Pages 文件处理：对页面层文件进行适配新目录结构的调整
手动完成	1. AI 逻辑优化：对 6 种 AI 行为分别调优，设置不同敌人的行为参数（蓄力时长、射击间隔、速度等），实现怪物行为多样化 2. 新增敌人：在 basic_enemy_data.h/.cpp 中添加了新敌人类型（Ogre、Jester、Ghost、Robot、XenoBeast 等） 3. Boss 类怪物：实现了 DemonLord、

	BoneLord、UFO、AlienPilot 四个 Boss 的多阶段战斗逻辑 4. 音频系统：筛选并导入 6 首 BGM 和 2 个音效文件，实现了按波次和 Boss 切换的配乐逻辑 5. 怪物波次管理完善：实现怪物刷新调度、波次递进、动态难度倍率等核心功能
--	--

V4 版本（手动 UI 打磨，AI 辅助稳定性修复）

维度	内容
手动完成	1. UI 全面优化：使用子窗口悬浮弹窗替代整页跳转，避免升级页面切换过于突兀；设计半透明 HUD 面板实现沉浸式战斗界面
AI 完成（Trae + DeepSeek-V4）	1. 信号链接优化：修复了因信号传导异常导致的游戏崩溃 Bug（V1 遗留代码导致的信号槽连接不稳定问题） 2. 构建部署：通过 Trae 构建 build-release 分发包，生成无需配置环境即可直接游玩的 .exe 可执行文件

9.3 各 AI 工具的分工说明

AI 工具	主要应用阶段	核心职责
豆包	V1 阶段	技术咨询、Qt 框架 API 和 C++ 语法答疑
ChatGPT 5.4	V2 阶段（通过 Trae 接入）	项目架构重构、工厂模式/MV 分离/多态实现、Bug 修复
DeepSeek-V4	V3-V4 阶段（通过 Trae 接入）	批量编码、文件重组、资源导入、信号修复、构建

9.4 文档撰写

- 本报告全文的格式排版、内容组织由人工主导，AI 辅助完成内容整合和润色
- 人工整理后，AI 进行图片生成
- CHANGELOG_V2.md、CHANGELOG_V3.md、V4 版本更新报告.md 等文档均由 AI 辅助生成初稿，人工审核和补充
- README 文档使用 AI 辅助生成

9.5 人工主导部分声明

以下核心工作完全由人工完成，未使用 AI 辅助：

- 游戏整体玩法设计、数值策划与平衡性调优
- 6 种 AI 行为模式的算法设计
- 像素级 UI 布局、色彩方案和视觉风格设计
- 所有 73 张 PNG 资源的筛选、裁剪、格式转换和 QRC 资源注册
- 6 首 BGM 的选择和波次配乐逻辑设计
- 最终数值平衡调优（经验阈值降低 25%、怪物密度与生成间隔调整等）
- 所有 AI 生成代码的人工逐行审查、逻辑修正和运行时验证

报告完成南开大学 计算机大类 2026 年 5 月 13 日